Customizing ArchLinux Installation Media

Rohit Goswami · 2023-04-30 · ramblings · setup · archlinux

> *A short plug for `hzArchiso`, and general thoughts on `archiso`*

Background

I recently had to switch server boxes, due to a cascade of hardware errors stemming from a remodeling of my home. The new box, however, did not see fit to include a WiFi card, which meant I had to add one of my own, a Realtek 88XXau, the kernel module for which is on the AUR.

This meant the official ArchLinux ISO would not be able to connect to the internet, and short of copying over files from PC to PC manually with a USB stick, my only option was to build my own installation media.

This then snowballed a little into a customized live-USB project called hzArchiso with a bunch of bells and whistles I need [^fn:1]. This post is not meant to supplant the ArchWiki and does not provide an in-depth introduction to using `archiso` [^fn:2].

Makepkg tweaks

When building system images, I tend to modify my `/etc/makepkg.conf` to use all cores and also use the faster `mold` linker (detailed on the ArchWiki):

```
MAKEFLAGS="-j$(nproc)"
GITFLAGS="--filter=tree:0"
LDFLAGS="... -fuse-ld=mold"
RUSTFLAGS="... -C link-arg=-fuse-ld=mold"
```

- **Do not use** `march=native` or other processor specific flags.
- `$GITFLAGS` support needs `pacman-git` as of July 2023, check the ArchWiki for details.

Managing AUR packages

`repoctl` (repo) is a fantastic tool for setting up local repositories of packages, which make it pretty trivial to add to an installation media as well. Since the repository is meant only for use with the installation media, it is sufficient to define it in only within the project, as discussed on the `archiso` ArchWiki entry.

Updating / adding packages can be done with:

```
## Once
repoctl conf new ~/pkgs/hzarchiso.db.tar.gz
repoctl reset
## Separate terminal
repoctl serve
## Everytime
mkdir tmpUpd && cd tmpUpd
for pkg in $(cat ../aurpkgs.txt); do
    (
        repoctl down "$pkg"
    )
done
for dir in *; do cd $dir; makepkg -cs && repoctl add *.pkg.tar.zst && cd ../  && rm -rf $dir || cd ../; done
cd ../ && rm -rf tmpUpd
```

Kernel modifications

It turns out that my new machine required a wireless card patch. I haven't compiled a kernel for several years now[^fn:3]. The process was gratifyingly simplistic, following some discussion threads on the Archlinux Forums.

Essentially:

- Grab the relevant patchset
- Add it to the `PKGBUILD`
- Build (optionally add to local repo)
- Profit

```
..
source = (
    "v2-wifi-iwlwifi-pcie-add-device-id51F1-for-killer-1675.patch::https://lore.kernel.org/linux-
wireless/20230608082725.353150-1-yi@yikuo.dev/raw"
)
..
b2sums=(

'e9f30d94b84a019c38842fbb468c64522782d14a347a12442704e185b4d9b131dc25661e7e9b9640ac20e0c949f5fa0375739d0c1badd
0a0c637408a56091118'
)
```

For building the `nitrous` kernel (an unofficial one for newer hardware) then:

```
repoctl down linux-nitrous
# modify the PKGBUILD
makepkg -si
```

Building kernels takes around as long as I remember, several hours at on an older laptop.

Using `hzArchiso`

The general idea is to follow along with the ArchWiki installation steps (perhaps augmented by the dual-booting instructions) For some users, the `archinstaller` script (repo) might be a better option. There's also an `offlineInst.sh` script which can be used to replicate the environment on the live-USB.

Conclusions

Modifying ArchLinux installation media is fun. Even if the exact set of packages bundled with `hzArchiso` isn't for most, it is a useful blueprint for branching out into their own ISOs. Pull requests and other issues / contributions are always welcome as well, and more detailed build information is on the repo and on the ArchWiki. Personally, I expect to update the `iso` every now and then with more of my configuration / defaults / every time I get a new machine. I've also been using `btrfs` for almost a decade without any trouble [^fn:4], which is anecdotal but nice.
